

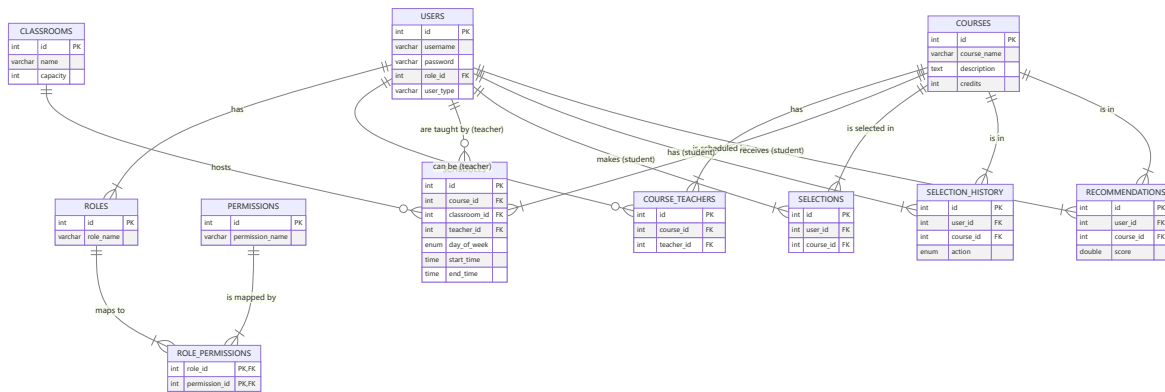
项目概要设计文档

1. 数据库设计

本系统的数据库旨在存储和管理课程、用户、选课、排课及推荐等核心数据。数据库采用关系型模型，以保证数据的一致性和完整性。

1.1 数据库 E-R 图

下面是系统的实体关系图（E-R 图），展示了核心实体及其相互关系。



1.2 数据库物理结构设计

以下是数据库中每个表的详细结构定义。

1. users - 用户表

存储所有用户（学生、教师、管理员）的基本信息。

```
CREATE TABLE `users` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `username` varchar(50) NOT NULL,  
  `password` varchar(255) NOT NULL,  
  `role_id` int DEFAULT NULL,  
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  `user_type` varchar(50) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `username` (`username`),  
  KEY `idx_users_role_id` (`role_id`),  
  CONSTRAINT `users_ibfk_1` FOREIGN KEY (`role_id`) REFERENCES `roles` (`id`) ON  
  DELETE SET NULL  
);
```

2. roles - 角色表

定义系统中的角色（ADMIN, TEACHER, STUDENT）。

```
CREATE TABLE `roles` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `role_name` varchar(50) NOT NULL,  
  `description` varchar(255) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `role_name` (`role_name`)  
);
```

3. permissions - 权限表

定义了具体的操作权限。

```
CREATE TABLE `permissions` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `permission_name` varchar(50) NOT NULL,  
  `description` varchar(255) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `permission_name` (`permission_name`)  
);
```

4. role_permissions - 角色权限关联表

将角色和权限进行多对多关联。

```
CREATE TABLE `role_permissions` (  
  `role_id` int NOT NULL,  
  `permission_id` int NOT NULL,  
  PRIMARY KEY (`role_id`, `permission_id`),  
  CONSTRAINT `role_permissions_ibfk_1` FOREIGN KEY (`role_id`) REFERENCES `roles`  
  (`id`) ON DELETE CASCADE,  
  CONSTRAINT `role_permissions_ibfk_2` FOREIGN KEY (`permission_id`) REFERENCES  
  `permissions` (`id`) ON DELETE CASCADE  
);
```

5. courses - 课程表

存储核心的课程信息。

```
CREATE TABLE `courses` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `course_name` varchar(100) NOT NULL,  
  `description` text,  
  `credits` int NOT NULL,  
  `weekly_hours` int NOT NULL,  
  `weekly_sessions` int NOT NULL,  
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`)  
);
```

6. `course_teachers` - 课程教师关联表

指定课程的授课教师。

```
CREATE TABLE `course_teachers` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `course_id` int NOT NULL,  
  `teacher_id` int NOT NULL,  
  `role` varchar(50) NOT NULL,  
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `course_teachers_ibfk_1` FOREIGN KEY (`course_id`) REFERENCES  
  `courses` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `course_teachers_ibfk_2` FOREIGN KEY (`teacher_id`) REFERENCES  
  `users` (`id`) ON DELETE CASCADE  
);
```

7. `classrooms` - 教室表

存储教室信息。

```
CREATE TABLE `classrooms` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `name` varchar(50) NOT NULL,  
  `capacity` int NOT NULL,  
  `location` varchar(100) DEFAULT NULL,  
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `name` (`name`)  
);
```

8. `schedules` - 排课表

存储课程的具体上课安排。

```
CREATE TABLE `schedules` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `course_id` int NOT NULL,  
  `classroom_id` int DEFAULT NULL,  
  `teacher_id` int DEFAULT NULL,  
  `day_of_week` enum('MON','TUE','WED','THU','FRI','SAT','SUN') NOT NULL,  
  `start_time` time NOT NULL,  
  `end_time` time NOT NULL,  
  `classroom` varchar(255) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `unique_schedule`  
  (`classroom_id`,`day_of_week`,`start_time`,`end_time`),  
  CONSTRAINT `schedules_ibfk_1` FOREIGN KEY (`course_id`) REFERENCES `courses`  
  (`id`) ON DELETE CASCADE,  
  CONSTRAINT `schedules_ibfk_2` FOREIGN KEY (`classroom_id`) REFERENCES  
  `classrooms` (`id`) ON DELETE SET NULL,
```

```
CONSTRAINT `schedules_ibfk_3` FOREIGN KEY (`teacher_id`) REFERENCES `users`  
(`id`) ON DELETE SET NULL  
);
```

9. selections - 学生选课表

记录学生当前的选课状态。

```
CREATE TABLE `selections` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `user_id` int NOT NULL,  
  `course_id` int NOT NULL,  
  `selected_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `unique_selection` (`user_id`, `course_id`),  
  CONSTRAINT `selections_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users`  
  (`id`) ON DELETE CASCADE,  
  CONSTRAINT `selections_ibfk_2` FOREIGN KEY (`course_id`) REFERENCES `courses`  
  (`id`) ON DELETE CASCADE  
);
```

10. selection_history - 选课历史表

记录所有的选课和退课操作，用于数据分析。

```
CREATE TABLE `selection_history` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `user_id` int NOT NULL,  
  `course_id` int NOT NULL,  
  `action` enum('SELECT', 'DROP') NOT NULL,  
  `timestamp` datetime DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `selection_history_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES  
  `users` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `selection_history_ibfk_2` FOREIGN KEY (`course_id`) REFERENCES  
  `courses` (`id`) ON DELETE CASCADE  
);
```

11. recommendations - 推荐结果表

存储为每个用户生成的课程推荐结果。

```
CREATE TABLE `recommendations` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `user_id` int NOT NULL,  
  `course_id` int NOT NULL,  
  `score` double NOT NULL,  
  `recommended_at` datetime DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `unique_recommendation` (`user_id`, `course_id`),  
  CONSTRAINT `recommendations_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users`  
  (`id`) ON DELETE CASCADE,  
  CONSTRAINT `recommendations_ibfk_2` FOREIGN KEY (`course_id`) REFERENCES  
  `courses` (`id`) ON DELETE CASCADE  
);
```

基于协同过滤算法的排课系统所使用的后端API接口。接口遵循RESTful设计原则，使用JSON格式进行数据交换，并通过HTTP状态码标识操作结果。

1. 通用约定

- **基地址:** `http://localhost:8080/api`
- **认证:** 大部分接口需要通过在请求头中携带 `Authorization: Bearer <token>` 进行认证。Token 通过用户登录接口获取。
- **响应格式:**
 - 成功: `{ "code": 200, "message": "Success", "data": { ... } }`
 - 失败: `{ "code": 4xx/5xx, "message": "Error description", "data": null }`

2. 认证接口

2.1 用户登录

- **功能:** 用户通过用户名和密码登录，获取认证Token。
- **路径:** `/auth/login`
- **方法:** `POST`
- **请求体** (`application/json`):

```
{  
  "username": "张三",  
  "password": "123456"  
}
```

- **成功响应** (200 OK):

```
{
  "code": 200,
  "message": "登录成功",
  "data": {
    "token": "eyJ...",
    "user": {
      "id": 4,
      "username": "张三",
      "role": "STUDENT"
    }
  }
}
```

3. 学生端接口

3.1 获取所有课程列表

- **功能:** 分页获取对学生开放的课程列表，可带查询参数。
- **路径:** `/student/courses`
- **方法:** GET
- **查询参数:**
 - `page` (可选, int): 页码，默认为 1。
 - `size` (可选, int): 每页数量，默认为 10。
 - `courseName` (可选, string): 课程名模糊查询。
- **成功响应** (200 OK):

```
{
  "code": 200,
  "message": "Success",
  "data": {
    "total": 25,
    "records": [
      {
        "id": 1,
        "courseName": "计算机科学与技术",
        "teacherName": "张老师",
        "credits": 3,
        "schedule": "周一 09:40-11:10 @ 教室A"
      }
    ]
  }
}
```

3.2 选课

- **功能:** 学生选择一门课程。
- **路径:** `/student/selections/{courseId}`
- **方法:** POST
- **路径参数:** `courseId` (int) - 要选择的课程ID。

- **成功响应** (200 OK):

```
{ "code": 200, "message": "选课成功" }
```

- **失败响应** (400 Bad Request):

```
{ "code": 400, "message": "选课失败: 时间冲突" }
```

```
{ "code": 400, "message": "选课失败: 课程容量已满" }
```

3.3 获取已选课程

- **功能:** 获取当前学生已选择的课程列表。
- **路径:** `/student/selections`
- **方法:** `GET`
- **成功响应** (200 OK):

```
{  
  "code": 200,  
  "message": "Success",  
  "data": [  
    { "selectionId": 11, "courseId": 16, "courseName": "公共数学", ... }  
  ]  
}
```

3.4 退课

- **功能:** 学生退出一门已选的课程。
- **路径:** `/student/selections/{courseId}`
- **方法:** `DELETE`
- **路径参数:** `courseId` (int) - 要退出的课程ID。
- **成功响应** (200 OK):

```
{ "code": 200, "message": "退课成功" }
```

3.5 获取个性化推荐课程

- **功能:** 获取系统为当前学生推荐的课程列表。
- **路径:** `/student/recommendations`
- **方法:** `GET`
- **成功响应** (200 OK):

```
{
  "code": 200,
  "message": "Success",
  "data": [
    { "courseId": 2, "courseName": "数据结构与算法", "score": 0.85, ... }
  ]
}
```

4. 教师端接口

4.1 教师开课

- **功能:** 教师创建一门新的课程。
- **路径:** `/teacher/courses`
- **方法:** `POST`
- **请求体:**

```
{
  "courseName": "高级软件工程",
  "description": "介绍高级软件开发方法...",
  "credits": 3,
  "weeklyHours": 3,
  "weeklySessions": 1
}
```

- **成功响应** (201 Created):

```
{
  "code": 201,
  "message": "课程创建成功",
  "data": { "courseId": 32, ... }
}
```

4.2 删除课程

- **功能:** 教师删除自己开设的课程。
- **路径:** `/teacher/courses/{courseId}`
- **方法:** `DELETE`
- **路径参数:** `courseId` (int) - 课程ID。
- **成功响应** (200 OK):

```
{ "code": 200, "message": "课程删除成功" }
```

- **失败响应** (400 Bad Request):

```
{ "code": 400, "message": "无法删除, 已有学生选择此课程" }
```


4.3 获取教师所授课程

- **功能:** 获取当前登录教师开设的所有课程列表。
- **路径:** `/teacher/courses`
- **方法:** `GET`
- **成功响应** (200 OK):

```
{
  "code": 200,
  "message": "Success",
  "data": [
    { "id": 1, "courseName": "计算机科学与技术", ... }
  ]
}
```

5. 管理员端接口

5.1 用户管理

- **获取用户列表:** `GET /admin/users`
- **创建新用户:** `POST /admin/users`

```
{ "username": "王五", "password": "...", "roleId": 3, "userType": "STUDENT" }
```

- **修改用户信息:** `PUT /admin/users/{userId}`
- **删除用户:** `DELETE /admin/users/{userId}`

5.2 课程管理

- **获取所有课程:** `GET /admin/courses`
- **修改课程信息:** `PUT /admin/courses/{courseId}`
- **删除任意课程:** `DELETE /admin/courses/{courseId}`

5.3 排课管理

- **为课程排课:** `POST /admin/schedules`

```
{
  "courseId": 1,
  "classroomId": 1,
  "teacherId": 2,
  "dayOfWeek": "MON",
  "startTime": "09:40:00",
  "endTime": "11:10:00"
}
```

- **获取所有排课信息:** `GET /admin/schedules`
- **删除排课信息:** `DELETE /admin/schedules/{scheduleId}`

5. 页面截图
